

UNIVERSITY OF LONDON

INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects



CM3060 NATURAL LANGUAGE PROCESSING MIDTERM COURSEWORK

SENTIMENTAL ANALYSIS ON UNIVERSAL STUDIOS THEME PARK REVIEWS: Comparing Naïve Bayes and BERT Models

Author : Devon Maya Xavior
UOL Student Number : 200618744
SIM Student Number : 10204322

TABLE OF CONTENTS

I. INTRODUCTION	3
A. Domain.....	3
B. Objectives.....	4
C. Dataset	5
D. Evaluation Methodology	6
Part 1: Classification	6
Part 2: Evaluation	6
Part 3: Metrics Relevance.....	6
Part 4: Validation	7
II. IMPLEMENTATION	7
A. Data Preprocessing	7
B. Naives Bayes Model	12
Train/Validation/Test Splits	14
C. BERT Model	21
III. CONCLUSION	29
A. Overall Performance Comparison	29
B. Class-Specific Performance Analysis.....	30
Negative Sentiment Detection	30
Positive Sentiment Detection	31
Samples of incorrectly Labelled sentiments by Naives Bayes	32
C. Statistical vs. Embedding-Based Model.....	33
IV. SUMMARY & REFLECTION	34
A. Learning Experience	34
B. Model Practicality	34
C. Application & Transferability	34
D. Contribution & Future Research	34
REFERENCES.....	35

I. INTRODUCTION

A. DOMAIN

Sentiment analysis in the **tourism and entertainment industry** has become increasingly critical for business decision-making, particularly in the theme park sector, where customer experience directly impacts revenue and brand reputation. **With over 50,000 customer reviews across Universal Studios' three global locations** (Florida, Japan, and Singapore), this type of text-based analysis can provide actionable insights for operational improvements, marketing strategies, and competitive positioning for this theme park franchise.

While traditional approaches rely heavily on numerical rating scores, **text-based sentiment analysis offers a more nuanced understanding of customer experiences**. Rating scores alone can be misleading: a 3-star rating might reflect either a genuinely mediocre experience or a good experience with one significant disappointment, while a 4-star rating may mask specific operational issues that customers explicitly mention in their written feedback. Text analysis captures the authentic Voice of the Customer (VOC) (Gallagher et al., 2019), revealing specific pain points such as poor service quality, inadequate parking facilities, or excessive wait times (Bai et al., 2023) that rating scores fail to communicate. This granular insight enables theme park management to take targeted corrective actions rather than relying on aggregate numerical trends.

But that does not mean there are no NLP challenges with this method of analysis. Customer reviews often contain mixed sentiments within single reviews, detailed narratives covering multiple aspects (rides, food service, staff interactions, wait times), domain-specific terminology and ride names, varying review lengths from brief complaints to comprehensive trip reports, and frequent comparative references to competing parks like Disney.

Therefore, comparing traditional statistical models like Naive Bayes with modern transformer-based models like BERT is particularly relevant for tourism businesses. **While statistical models offer interpretability and faster processing suitable for real-time customer service applications, deep learning models may better capture the contextual nuances and complex sentiment patterns inherent in detailed customer narratives.**

B. OBJECTIVES

The primary objective of this project is to compare the effectiveness of a traditional statistical model (**Naive Bayes**) and a modern transformer-based model (BERT) for sentiment classification of **Universal Studios theme park reviews**.

Specifically, this study aims to:

- **Assess classification accuracy:**
by comparing predicted sentiments against actual customer rating scores (1-5 stars) to determine which model better captures true customer sentiment
- **Analyse the processing speed versus accuracy trade-offs:**
to understand practical deployment considerations for real-time business applications
- **Determine which model provides more actionable business insights:**
by examining the interpretability and specificity of sentiment predictions for operational decision-making

For the purpose of this study, we will be referencing **Universal Studios** when talking about the potential impact the results will bring:

For Universal Studios stakeholders, including management, marketing, operations, and experience development teams, this analysis will produce meaningful insights that inform business strategies by systematically examining customer reviews to gauge satisfaction levels and pinpoint specific areas for improving the customer experience.

For sentiment analysis research in tourism, this study will serve as a comparative benchmark for similar investigations, particularly those examining theme park customer feedback (Bai et al., 2023) or utilizing Universal Studios TripAdvisor data (Niu et al., 2019) for cross-validation purposes.

Additionally, the findings will provide practical guidance **for tourism and entertainment businesses** when selecting between statistical and deep learning approaches for sentiment analysis, considering factors such as dataset size, computational resources, interpretability requirements, and real-time processing needs.

C. DATASET

The dataset utilized for this comparative sentiment analysis was sourced from **Kaggle** (Nurdialit, 2021) and consists of customer reviews originally collected from **TripAdvisor** through web scraping methodology. The dataset encompasses **50,904 customer reviews** spanning Universal Studios' three major global locations: **Universal Studios Florida, Universal Studios Singapore, and Universal Studios Japan**.

The dataset is structured as a comma-separated values (CSV) file containing six distinct columns:

- **Reviewer:** String data type containing the account name/username of the review author
- **Rating:** Integer values ranging from 1 (highly unsatisfied) to 5 (highly satisfied) representing customer satisfaction scores
- **Written Date:** Date format indicating when each review was posted on TripAdvisor
- **Title:** String data containing the headline or summary title of each review
- **Review Text:** String data representing the full narrative content of customer reviews, varying significantly in length and detail
- **Branch:** Categorical string data identifying the specific Universal Studios location (Florida, Singapore, or Japan)

This dataset is particularly well-suited for comparing Naive Bayes and BERT models for several reasons. The **substantial size of 50,904 reviews provides sufficient data** for training both statistical and deep learning models while enabling robust testing and validation. The **five-point rating scale (1-5)** serves as **reliable ground truth labels for supervised learning**, allowing objective performance evaluation. The presence of **mixed sentiment reviews and varying narrative lengths** creates the complexity needed to meaningfully test each model's ability to handle nuanced customer feedback. Furthermore, the **multi-location structure across three different cultural contexts (US, Singapore, Japan)** provides diversity in language patterns and cultural expressions, enhancing the generalizability of findings. Finally, the **real-world business context** ensures that model performance insights will have practical applicability for tourism industry stakeholders.

However, **several limitations** should be acknowledged. First, **the dataset was last updated four years ago**, which may limit the relevance of findings to current operational conditions and customer expectations, particularly given the significant changes in the tourism industry following the COVID-19 pandemic. Second, **the dataset exhibits inherent reviewer bias**, as individuals who post online reviews tend to represent extreme experiences (either highly positive or highly negative), potentially underrepresenting moderate customer opinions. Additionally, the reviews are likely **predominantly in English**, which **may not fully capture the sentiment patterns of international visitors** who prefer to review in their native languages, particularly for Universal Studios Japan.

D. EVALUATION METHODOLOGY

PART 1: CLASSIFICATION

This study will treat sentiment analysis as a supervised multi-class classification problem, where customer rating scores (1-5 stars) serve as ground truth labels for training and evaluation purposes.

PART 2: EVALUATION

Each model will be assessed using four standard classification metrics:

- **Accuracy** measures the overall proportion of correctly classified reviews across all sentiment categories (1-5 stars), providing a general indicator of model performance.
- **Precision** evaluates the model's ability to avoid false positives by measuring the proportion of predicted positive sentiments that are actually positive, which is crucial for Universal Studios to avoid misinterpreting negative feedback as positive.
- **Recall (Sensitivity)** assesses the model's ability to identify all actual positive or negative sentiments, ensuring that genuine customer complaints are not missed, which could result in unaddressed operational issues.
- **F1-Score** provides the mean of precision and recall, offering a balanced metric that is particularly valuable when dealing with potentially imbalanced datasets where certain rating categories (e.g., 5-star reviews) may be more frequent than others.

PART 3: METRICS RELEVANCE

For Universal Studios management, recall may be the most critical metric, as failing to identify genuine negative sentiment (false negatives) could result in unaddressed operational issues that compound over time and damage brand reputation. Missing a customer complaint about safety concerns or poor service is more costly than occasionally flagging a neutral review as negative. However, excessively low precision would waste management resources by triggering unnecessary investigations into false positive cases.

The F1-Score becomes particularly valuable for comparing Naive Bayes and BERT models, as it balances both precision and recall, ensuring that neither model gains an unfair advantage by optimizing only one metric. Additionally, given that customer review datasets often exhibit class imbalance (with more moderate to positive ratings than extremely negative ones), accuracy alone may be misleading and could favour models that simply predict the majority class.

PART 4: VALIDATION

The dataset will be **split into training (70%), validation (15%), and test (15%) sets** using stratified sampling to maintain proportional representation of all rating categories across splits.

Both Naive Bayes and BERT models will be **trained and evaluated on identical data splits** to guarantee fair comparison. The same preprocessing steps will be applied consistently, though model-specific text representations will differ (bag-of-words/TF-IDF for Naive Bayes versus tokenized sequences for BERT).

II. IMPLEMENTATION

A. DATA PREPROCESSING

We will first explore the dataset to see a sample of the content and how it is structured.

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv('universal_studio_branches.csv')

# Data exploration
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nFirst few rows:")
df.head()
```

Dataset shape: (50904, 6)

Column names:
['reviewer', 'rating', 'written_date', 'title', 'review_text', 'branch']

First few rows:

	reviewer	rating	written_date	title	review_text	branch
0	Kelly B	2.0	May 30, 2021	Universal is a complete Disaster - stick with ...	We went to Universal over Memorial Day weekend...	Universal Studios Florida
1	Jon	1.0	May 30, 2021	Food is hard to get.	The food service is horrible. I'm not reviewin...	Universal Studios Florida
2	Nerdy P	2.0	May 30, 2021	Disappointed	I booked this vacation mainly to ride Hagrid m...	Universal Studios Florida
3	ran101278	4.0	May 29, 2021	My opinion	When a person tries the test seat for the ride...	Universal Studios Florida
4	tammies20132015	5.0	May 28, 2021	The Bourne Stuntacular...MUST SEE	Ok, I can't stress enough to anyone and everyo...	Universal Studios Florida

Then, we will check for missing values and take a look at its rating distribution.

```
# Check for missing values
print("\nMissing values:")
print(df.isnull().sum())

# Look at rating distribution
print("\nRating distribution:")
print(df['rating'].value_counts().sort_index())
```

Missing values:

reviewer	0
rating	0
written_date	0
title	0
review_text	0
branch	0

dtype: int64

Rating distribution:

rating	
1.0	1973
2.0	1986
3.0	5229
4.0	13514
5.0	28202

Name: count, dtype: int64

From the results, we can see that the dataset has no missing values in all the columns, and also the patterns in the review ratings—positive reviews, i.e., 4- and 5-star, take dominance, whereas neutral (3-star) and negative (2- and 1-star) reviews are significantly lower compared to the positive ones. This indicates a clear imbalance in the distribution of review ratings.

We will then proceed to find out the branch distribution and explore the textual characteristics of the content itself to see the review length and also some sample text.

```
# Branch distribution
print("Branch distribution:")
print(df['branch'].value_counts())
```

Branch distribution:

branch	
Universal Studios Florida	30621
Universal Studios Singapore	15754
Universal Studios Japan	4529

Name: count, dtype: int64


```
# Review text lengths
df['text_length'] = df['review_text'].str.len()
print("\nReview text length statistics:")
print(df['text_length'].describe())

# Sample review texts by rating
print("\nSample reviews by rating:")
for rating in [1, 3, 5]:
    print(f"\n--- Rating {rating} example ---")
    sample = df[df['rating'] == rating]['review_text'].iloc[0]
    print(sample[:200] + "... " if len(sample) > 200 else sample)
```

```
Review text length statistics:
count      50904.000000
mean        629.729589
std         733.427170
min          40.000000
25%         227.000000
50%         406.000000
75%         749.000000
max        19114.000000
Name: text_length, dtype: float64
```

--- Rating 1 example ---

The food service is horrible. I'm not reviewing the food. It's the wait time!!!! 45 minutes minimum to get to the one cashier working at each place. Ruins the whole experience. 5 closed cash registers...

--- Rating 3 example ---

Many fun simulator rides and I love the "Wizarding World," but overall customer service stinks. Specifically, couldn't get in early because only 1 gate was open, even though I stayed at a Universal re...

--- Rating 5 example ---

Ok, I can't stress enough to anyone and everyone... if you go to Universal Studios Orlando... make sure you take the time to go see The Bourne Stuntacular...I had to watch it 3xs and it wasn't enough....

From the above 2 outputs, it shows that the majority of the reviews were for Florida, which makes sense as it is the oldest, therefore most visited, followed by Singapore and lastly, Japan.

As for text lengths, it averages at 629 characters; the shortest reviews are at 40 characters, and the longest reviews can go up to 19,114 characters (which can either indicate an extremely good or bad experience), but most reviews are in the range of 227 to 749 characters long.

The review samples already give a snippet of negative sentiments towards the food and customer service, but the third review sample is a positive one, "stressing" the need for park-goers to see The Bourne Stuntacular attraction.

As this project is focusing on sentiment analysis and not predictions for 5-class ratings, we will skip the 3-star rating data in order to create binary positive/negative labels.

```
# 1-2 as negative, 4-5 as positive, and skip 3 for now
df['sentiment'] = df['rating'].map({
    1.0: 'negative',
    2.0: 'negative',
    4.0: 'positive',
    5.0: 'positive'
})

# Remove neutral reviews (rating 3) for clearer sentiment classification
df_sentiment = df[df['rating'] != 3.0].copy()

print(f"After removing neutral reviews:")
print(f"Original dataset: {len(df)} reviews")
print(f"Sentence dataset: {len(df_sentiment)} reviews")
print("\nSentiment distribution:")
print(df_sentiment['sentiment'].value_counts())
```

```
After removing neutral reviews:
Original dataset: 50904 reviews
Sentiment dataset: 45675 reviews
```

```
Sentiment distribution:
sentiment
positive    41716
negative     3959
Name: count, dtype: int64
```

There is a severe imbalance between the newly labelled positive and negative reviews, which is actually rather realistic, as most customers tend to leave positive reviews. This further substantiates the need for F1-scoring later, as a model that always predicts "positive" will get 91% accuracy in our case.

I will now move on to basic text cleaning processes.

```
import re
import string

# Basic text cleaning function
def clean_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove special characters and digits
    text = re.sub(r'^a-zA-Z\s', '', text)
    # Remove extra whitespace
    text = ' '.join(text.split())
    return text

# Apply cleaning to a sample
print("Before cleaning:")
print(df_sentiment['review_text'].iloc[0][:200])

# Clean the text
df_sentiment['clean_text'] = df_sentiment['review_text'].apply(clean_text)

print("\nAfter cleaning:")
print(df_sentiment['clean_text'].iloc[0][:200])
```

Before cleaning:

We went to Universal over Memorial Day weekend and it was a total train wreck. We waited to get in the parking lot for about forty minutes. We paid for prime parking to make up for all the wasted time

After cleaning:

we went to universal over memorial day weekend and it was a total train wreck we waited to get in the parking lot for about forty minutes w
e paid for prime parking to make up for all the wasted time t

The text has now been cleaned of its uppercasing, punctuation has been removed, and extra whitespace removed. The next crucial step is to preprocess the text according to what is needed for each specific model.

B. NAIVES BAYES MODEL

For the Naive Bayes model, it treats words consistently, but we have already taken care of that by removing uppercasing, which will reduce vocabulary size and result in fewer unique tokens for more efficient processing. The next step now is to remove common stopwords that do not carry sentiment meaning yet take up most of the word frequency. This leaves us with more meaningful tokens to extract sentiments from. We are also removing short words (i.e., length less than or equal to 2) as they usually do not carry much sentiment. The preprocessing stage for the BERT model will be rather different later on because the latter actually needs stopwords and punctuation as it understands the context and relationships between words.

We will first import the **TfidfVectorizer** that will convert text into numbers that the Naives Bayes model can understand, as it only works with numbers and not text. It will create a "bag of words" matrix where each review will become a row of numbers.

The second import is **train_test_split**, which we will use for both models to split the dataset into training, validation, and testing portions.

The third import is the **nltk** package that will allow us to perform the removal of stopwords as aforementioned.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

# Preprocessing for Naive Bayes
import nltk
# nltk.download('stopwords')
from nltk.corpus import stopwords

# English stopwords
stop_words = set(stopwords.words('english'))

# Naive Bayes preprocessing function
def preprocess_for_nb(text):
    # Remove stopwords
    words = text.split()
    words = [word for word in words if word not in stop_words and len(word) > 2]
    return ' '.join(words)

# Apply NB preprocessing
df_sentiment['nb_text'] = df_sentiment['clean_text'].apply(preprocess_for_nb)

print("Naive Bayes preprocessing example:")
print("Original:", df_sentiment['clean_text'].iloc[0][:150])
print("NB version:", df_sentiment['nb_text'].iloc[0][:150])

```

Naive Bayes preprocessing example:

Original: we went to universal over memorial day weekend and it was a total train wreck we waited to get in the parking lot for about forty minutes we paid for

NB version: went universal memorial day weekend total train wreck waited get parking lot forty minutes paid prime parking make wasted time paid extra express pass

We can see from the preprocessed example output that stopwords have been successfully removed. Key sentiment words like "train wreck," "waited," and "wasted" have been kept, which will be very useful for the analysis later.

TRAIN/VALIDATION/TEST SPLITS

We will proceed to split the dataset into training (70%), validation (15%), and test (15%) sets, which will be the same ratio used for both models.

```
# Train/Validation/Test splits (70/15/15)
X = df_sentiment['review_text']
y = df_sentiment['sentiment']

# First split: 70% train, 30% temp
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Second split: 30% temp → 15% validation, 15% test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.5,
    random_state=42,
    stratify=y_temp
)

print("Train/Validation/Test Split Results:")
print(f"Training samples: {len(X_train)} ({len(X_train)/len(X)*100:.1f}%)")
print(f"Validation samples: {len(X_val)} ({len(X_val)/len(X)*100:.1f}%)")
print(f"Testing samples: {len(X_test)} ({len(X_test)/len(X)*100:.1f}%)")

print(f"\nTraining sentiment distribution:")
train_counts = y_train.value_counts()
print(train_counts)
for sentiment in ['positive', 'negative']:
    count = train_counts[sentiment]
    percentage = (count / len(y_train)) * 100
    print(f"{sentiment}: {count} ({percentage:.1f}%)")

print(f"\nValidation sentiment distribution:")
val_counts = y_val.value_counts()
print(val_counts)
for sentiment in ['positive', 'negative']:
    count = val_counts[sentiment]
    percentage = (count / len(y_val)) * 100
    print(f"{sentiment}: {count} ({percentage:.1f}%)")

print(f"\nTesting sentiment distribution:")
test_counts = y_test.value_counts()
print(test_counts)
for sentiment in ['positive', 'negative']:
    count = test_counts[sentiment]
    percentage = (count / len(y_test)) * 100
    print(f"{sentiment}: {count} ({percentage:.1f}%)")
```

Train/Validation/Test Split Results:

Training samples: 31972 (70.0%)
Validation samples: 6851 (15.0%)
Testing samples: 6852 (15.0%)

Training sentiment distribution:

```
sentiment
positive    29201
negative     2771
Name: count, dtype: int64
positive: 29201 (91.3%)
negative: 2771 (8.7%)
```

Validation sentiment distribution:

```
sentiment
positive     6257
negative      594
Name: count, dtype: int64
positive: 6257 (91.3%)
negative: 594 (8.7%)
```

Testing sentiment distribution:

```
sentiment
positive     6258
negative      594
Name: count, dtype: int64
positive: 6258 (91.3%)
negative: 594 (8.7%)
```

We have split the data successfully into their respective sets and maintained the same positive/negative ratio for each set as well. The next step will be to create TF-IDF Vectors for Naive Bayes.

```
# Get preprocessed text for train/validation/test sets
X_train_nb = df_sentiment.loc[X_train.index, 'nb_text']
X_val_nb = df_sentiment.loc[X_val.index, 'nb_text']
X_test_nb = df_sentiment.loc[X_test.index, 'nb_text']

# Create TF-IDF vectorizer
tfidf = TfidfVectorizer(
    max_features=5000,    # limit vocabulary to top 5000 words
    min_df=2,            # word must appear in at least 2 documents
    max_df=0.8           # ignore words in more than 80% of documents
)

# Fit on training data and transform all three sets
X_train_tfidf = tfidf.fit_transform(X_train_nb)
X_val_tfidf = tfidf.transform(X_val_nb)
X_test_tfidf = tfidf.transform(X_test_nb)
```

```

print("TF-IDF Vectorization Results:")
print(f"Vocabulary size: {len(tfidf.vocabulary_)}")
print(f"Training matrix shape: {X_train_tfidf.shape}")
print(f"Validation matrix shape: {X_val_tfidf.shape}")
print(f"Testing matrix shape: {X_test_tfidf.shape}")

# Example features
feature_names = tfidf.get_feature_names_out()
print(f"\nSample vocabulary words: {feature_names[:20].tolist()}")

```

TF-IDF Vectorization Results:

Vocabulary size: 5000

Training matrix shape: (31972, 5000)

Validation matrix shape: (6851, 5000)

Testing matrix shape: (6852, 5000)

Sample vocabulary words: ['aaa', 'ability', 'abit', 'able', 'absolute', 'absolutely', 'abundance', 'accelerator', 'accept', 'acceptable', 'accepted', 'access', 'accessibility', 'accessible', 'accident', 'accidentally', 'accommodate', 'accommodating', 'accommodation', 'accommodations']

The TF-IDF results show that our vocabulary size is now at 5000. Now that preprocessing for Naives Bayes is done, we will proceed to implement the model on the processed data.

We will first import **MultinomialNB** to implement the Naives Bayes algorithm that works well with word count data like the TF-IDF vectors we just created. It will calculate the probability that a review is positive/negative based on the word frequency.

Next, the **4 metric functions** mentioned under "Evaluation Methodology" will be imported as well (i.e., `accuracy_score`, `precision_score`, `recalled_score`, and `f1_score`).

Finally, we will implement the Naives Bayes model steps to get the **baseline performance** so we can compare it against the BERT model afterwards.


```

from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

# Create and train Naive Bayes model
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)

# Make predictions
y_pred_nb = nb_model.predict(X_test_tfidf)

# Calculate metrics
nb_accuracy = accuracy_score(y_test, y_pred_nb)
nb_precision = precision_score(y_test, y_pred_nb, pos_label='positive')
nb_recall = recall_score(y_test, y_pred_nb, pos_label='positive')
nb_f1 = f1_score(y_test, y_pred_nb, pos_label='positive')

print("Naive Bayes Results:")
print(f"Accuracy: {nb_accuracy:.3f}")
print(f"Precision: {nb_precision:.3f}")
print(f"Recall: {nb_recall:.3f}")
print(f"F1-Score: {nb_f1:.3f}")

```

Naive Bayes Results:
Accuracy: 0.933
Precision: 0.934
Recall: 0.996
F1-Score: 0.964

From the scores, we can see that:

- **Accuracy:** The Naives Bayes model managed to predict 93 out of every 100 predictions correctly. Which, in other words, means that if Universal Studios processed 1000 reviews, 933 would be classified correctly and 67 would be wrong, which is quite an impressive number.
- **Precision:** The Naives Bayes model managed to predict "positive" results with 93.4% precision. So that means 66 out of 1000 "positive" reviews are false; they are actually unhappy customers that might get overlooked.
- **Recall:** The recall score shows that of all truly positive reviews, the model is able to catch 99.6% of them, with 0.4% of positive reviews getting wrongly labeled as negative.
- **F1-Score:** The score of 96.4% shows a balance between precision and recall, which implies that the Naives Bayes model is consistently good at the above two assessments.

However, there is a possibility that the high accuracy rates we are seeing here might be misleading. Since we started off with 91.3% positive reviews in the preprocessed data, a model that always guesses "positive" would get 91.3% accuracy. Therefore, we will need to see how well it catches **negative reviews** to get a better idea of how well this model works.

```

# Negative class performance
from sklearn.metrics import precision_score, recall_score, f1_score

nb_precision_neg = precision_score(y_test, y_pred_nb, pos_label='negative')
nb_recall_neg = recall_score(y_test, y_pred_nb, pos_label='negative')
nb_f1_neg = f1_score(y_test, y_pred_nb, pos_label='negative')

print(f"\nNAIVE BAYES - NEGATIVE CLASS PERFORMANCE:")
print("="*50)
print(f"Precision (negative): {nb_precision_neg:.3f}")
print(f"Recall (negative): {nb_recall_neg:.3f}")
print(f"F1-Score (negative): {nb_f1_neg:.3f}")

# Business interpretation
total_negative_reviews = (y_test == 'negative').sum()
detected_negative_reviews = int(nb_recall_neg * total_negative_reviews)
missed_negative_reviews = total_negative_reviews - detected_negative_reviews

print(f"\nBUSINESS IMPACT:")
print(f"Total negative reviews: {total_negative_reviews}")
print(f"Detected by Naive Bayes: {detected_negative_reviews}")
print(f"Missed complaints: {missed_negative_reviews}")
print(f"Miss rate: {(1 - nb_recall_neg)*100:.1f}%")

```

NAIVE BAYES – NEGATIVE CLASS PERFORMANCE:

```

=====
Precision (negative): 0.876
Recall (negative): 0.261
F1-Score (negative): 0.402

```

BUSINESS IMPACT:

```

Total negative reviews: 594
Detected by Naive Bayes: 155
Missed complaints: 439
Miss rate: 73.9%

```

The results show pretty bad performance at catching negative reviews from the data. As the accuracy metric is only for the whole model, not for each class (i.e., positive/negative), the precision, recall and F1-score will be the indicators of performance. The support column is to show the true count of each class.

- **Precision:** The precision score of 87.6% shows that the model is usually right when it predicts a negative review, which is not too bad.
- **Recall:** Unfortunately, the recall score of 26.1% indicates that of all truly "negative" reviews, it is only right 26 out of every 100 it processed.
- **F1-Score:** Similarly, the f1-score score is low at 40.2%, which means that both precision and recall are not performing well.

The implications of the above results mean that the Naives Bayes model is really bad at detecting negative reviews; out of 594 negative reviews, the model only detects 155. Its miss rate is high at 73.9%. If Universal Studios were to use this model, out of 100 truly unhappy customers, it would only be able to discover 26; the remaining 74 would be falsely labeled as happy. The management will think that most of their customers are happy when lots of them are actually frustrated.

The reason for this bad performance is probably due to the class imbalance between positive and negative data; because of that, when the model is unsure, it will guess positive because it is right 91.3% of the time.

Let us do some evaluation on the validation and test sets too before moving on to BERT.

```
# Test Naive Bayes on Validation Set
print("NAIVE BAYES - VALIDATION SET EVALUATION")
print("="*50)

# Get predictions on validation set
y_pred_nb_val = nb_model.predict(X_val_tfidf)

# Calculate validation metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

nb_val_accuracy = accuracy_score(y_val, y_pred_nb_val)
print(f"Validation Accuracy: {nb_val_accuracy:.3f}")
print("\nValidation Classification Report:")
print(classification_report(y_val, y_pred_nb_val, target_names=['Negative', 'Positive']))

# Test Naive Bayes on Test Set
print("\nNAIVE BAYES - TEST SET EVALUATION")
print("="*50)

# Get predictions on test set
y_pred_nb_test = nb_model.predict(X_test_tfidf)

# Calculate test metrics
nb_test_accuracy = accuracy_score(y_test, y_pred_nb_test)
print(f"Test Accuracy: {nb_test_accuracy:.3f}")
print("\nTest Classification Report:")
print(classification_report(y_test, y_pred_nb_test, target_names=['Negative', 'Positive']))

# Compare Validation vs Test Performance
print("\nNAIVE BAYES - VALIDATION vs TEST COMPARISON")
print("="*50)
print(f"{'Metric':<20} {'Validation':<12} {'Test':<12} {'Difference':<12}")
print("-" * 56)
print(f"{'Accuracy':<20} {nb_val_accuracy:<12.3f} {nb_test_accuracy:<12.3f} {abs(nb_val_accuracy - nb_test_accuracy):<12.3f}")

# Store results for later BERT comparison
nb_val_report = classification_report(y_val, y_pred_nb_val, target_names=['Negative', 'Positive'], output_dict=True)
nb_test_report = classification_report(y_test, y_pred_nb_test, target_names=['Negative', 'Positive'], output_dict=True)

print(f"\nValidation Negative Recall: {nb_val_report['Negative']['recall']:.3f}")
print(f"Test Negative Recall: {nb_test_report['Negative']['recall']:.3f}")
```

NAIVE BAYES – VALIDATION SET EVALUATION

Validation Accuracy: 0.930

Validation Classification Report:

	precision	recall	f1-score	support
Negative	0.86	0.23	0.36	594
Positive	0.93	1.00	0.96	6257
accuracy			0.93	6851
macro avg	0.89	0.61	0.66	6851
weighted avg	0.92	0.93	0.91	6851

NAIVE BAYES – TEST SET EVALUATION

Test Accuracy: 0.933

Test Classification Report:

	precision	recall	f1-score	support
Negative	0.88	0.26	0.40	594
Positive	0.93	1.00	0.96	6258
accuracy			0.93	6852
macro avg	0.90	0.63	0.68	6852
weighted avg	0.93	0.93	0.92	6852

NAIVE BAYES – VALIDATION vs TEST COMPARISON

Metric	Validation	Test	Difference
Accuracy	0.930	0.933	0.003

Validation Negative Recall: 0.229

Test Negative Recall: 0.261

From the results, we can see there is no overfitting; there is only a tiny difference of 0.3%; both results are very close to the results we obtained from the training set. So we can use the results from Naives Bayes training set as the baseline to compare with BERT.

C. BERT MODEL

As mentioned earlier during the Naives Bayes model preprocessing, the steps for the BERT model are different, as it needs the original text with punctuation and context—we removed punctuation for the Naives Bayes model.

To proceed, we will first install and import the necessary BERT libraries and create the necessary datasets.

- **Transformers:** This is the Hugging Face library that provides pre-trained BERT models. It provides pre-trained weights, tokenizers, and training utilities.
- **Torch:** This is a deep learning framework that handles matrix operations, gradients, and model training that BERT requires.
- **AutoTokenizer:** It is a pre-trained BERT model + classification head for sentiment analysis. We are using this because it is built for text classification (instead of question answering or translation). It contains 66 million parameters that were trained on massive text data (using DistilBERT, a distilled version of BERT that maintains 97% of BERT's performance while being 60% smaller).
- **Training Arguments & Trainer:** These two imports configure how to train (i.e., learning rate, batch size, epochs) and handle the actual training process, respectively.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
import torch
from torch.utils.data import Dataset
import numpy as np

print("BERT imports successful")
```

BERT imports successful

```
# Import Dataset from datasets library
from datasets import Dataset

# Disable widgets to prevent display errors
import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"
os.environ["DISABLE_TQDM"] = "true"

# Also disable datasets progress bars
import datasets
datasets.disable_progress_bar()

# Load BERT tokenizer
tokenizer = AutoTokenizer.from_pretrained('distilbert-base-uncased')
```

```

# Use basic clean text for all three sets
X_train_bert = df_sentiment.loc[X_train.index, 'clean_text'].tolist()
X_val_bert = df_sentiment.loc[X_val.index, 'clean_text'].tolist()
X_test_bert = df_sentiment.loc[X_test.index, 'clean_text'].tolist()

# Convert labels to numbers for BERT
label_map = {'negative': 0, 'positive': 1}
y_train_bert = [label_map[label] for label in y_train]
y_val_bert = [label_map[label] for label in y_val]
y_test_bert = [label_map[label] for label in y_test]

print("BERT Data Preparation:")
print(f"Training samples: {len(X_train_bert)}")
print(f"Validation samples: {len(X_val_bert)}")
print(f"Testing samples: {len(X_test_bert)}")
print(f"Sample training text: {X_train_bert[0][:100]}...")
print(f"Sample training label: {y_train_bert[0]} (0=negative, 1=positive)")

# Create datasets for BERT using HuggingFace Dataset
train_dataset = Dataset.from_dict({
    'text': X_train_bert,
    'labels': y_train_bert
})

val_dataset = Dataset.from_dict({
    'text': X_val_bert,
    'labels': y_val_bert
})

test_dataset = Dataset.from_dict({
    'text': X_test_bert,
    'labels': y_test_bert
})

print(f"\nBERT Datasets Created:")
print(f"Train dataset: {len(train_dataset)} samples")
print(f"Validation dataset: {len(val_dataset)} samples")
print(f"Test dataset: {len(test_dataset)} samples")

# Add tokenization to datasets
def tokenize_function(examples):
    return tokenizer(examples['text'], truncation=True, padding='max_length', max_length=128)

train_dataset = train_dataset.map(tokenize_function, batched=True)
val_dataset = val_dataset.map(tokenize_function, batched=True)
test_dataset = test_dataset.map(tokenize_function, batched=True)

print("Datasets tokenized and ready for BERT evaluation")

```

BERT Data Preparation:
Training samples: 31972
Validation samples: 6851
Testing samples: 6852
Sample training text: im pregnant although there is not a lot i can do rides wise there is still a lot to do experience an...
Sample training label: 1 (0=negative, 1=positive)

BERT Datasets Created:
Train dataset: 31972 samples
Validation dataset: 6851 samples
Test dataset: 6852 samples
Datasets tokenized and ready for BERT evaluation

We can see from the results that the same train/test split ratio as Naives Bayes has been used for a fair comparison. The sample training text is a full sentence, keeping the context and labelling them with binary numbers (i.e., 0=negative, 1=positive).

With the BERT datasets created, we will set up some training arguments for the model and proceed to train it.


```

import time

model_name = 'distilbert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)

# Training arguments
training_args = TrainingArguments(
    output_dir='./distilbert_results',
    num_train_epochs=2,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    warmup_steps=500,
    weight_decay=0.01,
    logging_steps=500,
    save_steps=1000,
    eval_steps=1000,
    disable_tqdm=True,
    report_to=[],
    logging_first_step=False,
)

# Create trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
)

print("DistilBERT trainer setup complete")
print("Starting training...")
start_time = time.time()

# Start training
training_results = trainer.train()

end_time = time.time()
training_minutes = (end_time - start_time) / 60
print(f"DistilBERT training completed in {training_minutes:.1f} minutes!")

# Display training metrics
print("\nTRAINING METRICS:")
print("="*40)
print(f"Final training loss: {training_results.training_loss:.4f}")
print(f"Training steps completed: {training_results.global_step}")
print(f"Training samples per second: {training_results.metrics['train_samples_per_second']:.2f}")
print(f"Training steps per second: {training_results.metrics['train_steps_per_second']:.3f}")

```



```

DistilBERT trainer setup complete
Starting training...
{'loss': 0.2268, 'grad_norm': 2.6179680824279785, 'learning_rate': 4.99e-05, 'epoch': 0.25012506253126565}
{'loss': 0.1558, 'grad_norm': 2.228412389755249, 'learning_rate': 4.2867352773013155e-05, 'epoch': 0.5002501250625313}
{'loss': 0.1259, 'grad_norm': 0.5474254488945007, 'learning_rate': 3.572041166380789e-05, 'epoch': 0.7503751875937968}
{'loss': 0.1176, 'grad_norm': 6.885007381439209, 'learning_rate': 2.8573470554602633e-05, 'epoch': 1.0005002501250626}
{'loss': 0.0678, 'grad_norm': 0.08767534792423248, 'learning_rate': 2.142652944539737e-05, 'epoch': 1.2506253126563283}
{'loss': 0.0769, 'grad_norm': 4.096128463745117, 'learning_rate': 1.427958833619211e-05, 'epoch': 1.500750375187594}
{'loss': 0.0731, 'grad_norm': 4.958443641662598, 'learning_rate': 7.132647226986849e-06, 'epoch': 1.7508754377188596}
{'train_runtime': 2392.2907, 'train_samples_per_second': 26.729, 'train_steps_per_second': 1.671, 'train_loss': 0.11341872556380118, 'epoch': 2.0}
DistilBERT training completed in 39.9 minutes!

```

TRAINING METRICS:

```

=====
Final training loss: 0.1134
Training steps completed: 3998
Training samples per second: 26.73
Training steps per second: 1.671

```

Let's take a look at the BERT training results.

The BERT training took about 39.9 minutes to complete, which is a lot longer than the time it took to train the Naïves Bayes model (i.e., less than a second). Although there was a steep initial learning curve, there was consistent improvement, with loss decreasing steadily from 0.2268 to 0.0731.

Now, let's see what results we will get from validation and test data.

```

# Evaluate BERT model on Validation and Test sets
print("EVALUATING DISTILBERT ON VALIDATION AND TEST SETS")
print("="*60)

# =====
# VALIDATION SET EVALUATION
# =====
print("\nVALIDATION SET EVALUATION:")
print("-" * 40)

# Get predictions on validation set
predictions_val = trainer.predict(val_dataset)
y_pred_bert_val = predictions_val.predictions.argmax(-1)

# Convert pandas series string labels to integers for comparison
y_val_numeric = y_val.map({'negative': 0, 'positive': 1})

print("Validation label conversion check:")
print("Original y_val sample:", y_val.head())
print("Converted y_val_numeric sample:", y_val_numeric.head())
print("BERT validation predictions sample:", y_pred_bert_val[:5])

```

```

# Calculate validation metrics
bert_val_accuracy = accuracy_score(y_val_numeric, y_pred_bert_val)
bert_val_report = classification_report(y_val_numeric, y_pred_bert_val, target_names=['Negative', 'Positive'], output_dict=True)

print(f"\nDistilBERT Validation Accuracy: {bert_val_accuracy:.3f}")
print("\nValidation Classification Report:")
print(classification_report(y_val_numeric, y_pred_bert_val, target_names=['Negative', 'Positive']))

# =====
# TEST SET EVALUATION
# =====
print("\nTEST SET EVALUATION:")
print("-" * 40)

# Get predictions on test set
predictions_test = trainer.predict(test_dataset)
y_pred_bert_test = predictions_test.predictions.argmax(-1)

# Convert pandas series string labels to integers for comparison
y_test_numeric = y_test.map({'negative': 0, 'positive': 1})

print("Test label conversion check:")
print("Original y_test sample:", y_test.head())
print("Converted y_test_numeric sample:", y_test_numeric.head())
print("BERT test predictions sample:", y_pred_bert_test[:5])

# Calculate test metrics
bert_test_accuracy = accuracy_score(y_test_numeric, y_pred_bert_test)
bert_test_report = classification_report(y_test_numeric, y_pred_bert_test, target_names=['Negative', 'Positive'], output_dict=True)

print(f"\nDistilBERT Test Accuracy: {bert_test_accuracy:.3f}")
print("\nTest Classification Report:")
print(classification_report(y_test_numeric, y_pred_bert_test, target_names=['Negative', 'Positive']))

```

EVALUATING DISTILBERT ON VALIDATION AND TEST SETS

VALIDATION SET EVALUATION:

```

Validation label conversion check:
Original y_val sample: 30444    positive
28685    positive
30412    positive
45298    positive
20960    positive
Name: sentiment, dtype: object
Converted y_val_numeric sample: 30444    1
28685    1
30412    1
45298    1
20960    1
Name: sentiment, dtype: int64
BERT validation predictions sample: [1 1 1 1 1]

```

DistilBERT Validation Accuracy: 0.966

```

Validation Classification Report:
              precision    recall  f1-score   support

   Negative         0.83         0.76         0.79         594
   Positive         0.98         0.99         0.98        6257

   accuracy                   0.97         6851
  macro avg         0.90         0.87         0.89         6851
 weighted avg         0.96         0.97         0.96         6851

```

TEST SET EVALUATION:

```

Test label conversion check:
Original y_test sample: 32972    negative
5812    positive
32989    positive
22498    positive
25510    positive
Name: sentiment, dtype: object
Converted y_test_numeric sample: 32972    0
5812    1
32989    1
22498    1
25510    1
Name: sentiment, dtype: int64
BERT test predictions sample: [1 1 1 1 1]

```

DistilBERT Test Accuracy: 0.966

```

Test Classification Report:
              precision    recall  f1-score   support

   Negative         0.84         0.75         0.79         594
   Positive         0.98         0.99         0.98        6258

   accuracy                   0.97         6852
  macro avg         0.91         0.87         0.89         6852
 weighted avg         0.97         0.97         0.97         6852

```

DISTILBERT – VALIDATION vs TEST COMPARISON

Metric	Validation	Test	Difference
Accuracy	0.966	0.966	0.001
Negative Recall	0.758	0.753	0.005
Positive Recall	0.985	0.987	0.001

Let's evaluate the BERT results:

Both validation and test results are almost exactly the same, with only a 0.1% to 0.5% difference.

We will look at the performance on positive reviews first:

Overall Accuracy: The BERT model has achieved an overall accuracy rate of 96.6%, which is a good indicator.

Precision: It has a 98% accuracy at detecting truly positive reviews successfully. Which means, when Universal Studios flags a review as a happy customer, they are 98% right all the time.

Recall: It scored 99% for its recall rate, indicating that it only misses 1% of actual positive reviews, and from the perspective of Universal Studios, this means only a pretty small number of happy customers' reviews get overlooked.

F1-Score: Again, a 98% score shows the BERT system is working consistently well at positive/negative detection.

Now for the performance on detecting negative reviews:

Precision: A score of 84% says that it does make mistakes with 16% of the negative reviews, which are supposed to be positive ones. So, 16% of the unhappy customers detected are actually happy ones.

Recall: Recall score is a little lower at 75%, so that means it can only correctly find 75% of the negative reviews and misses 25% of them. From the angle of Universal Studios, that shows that out of every 100 unhappy customers, they could only detect 75 of them.

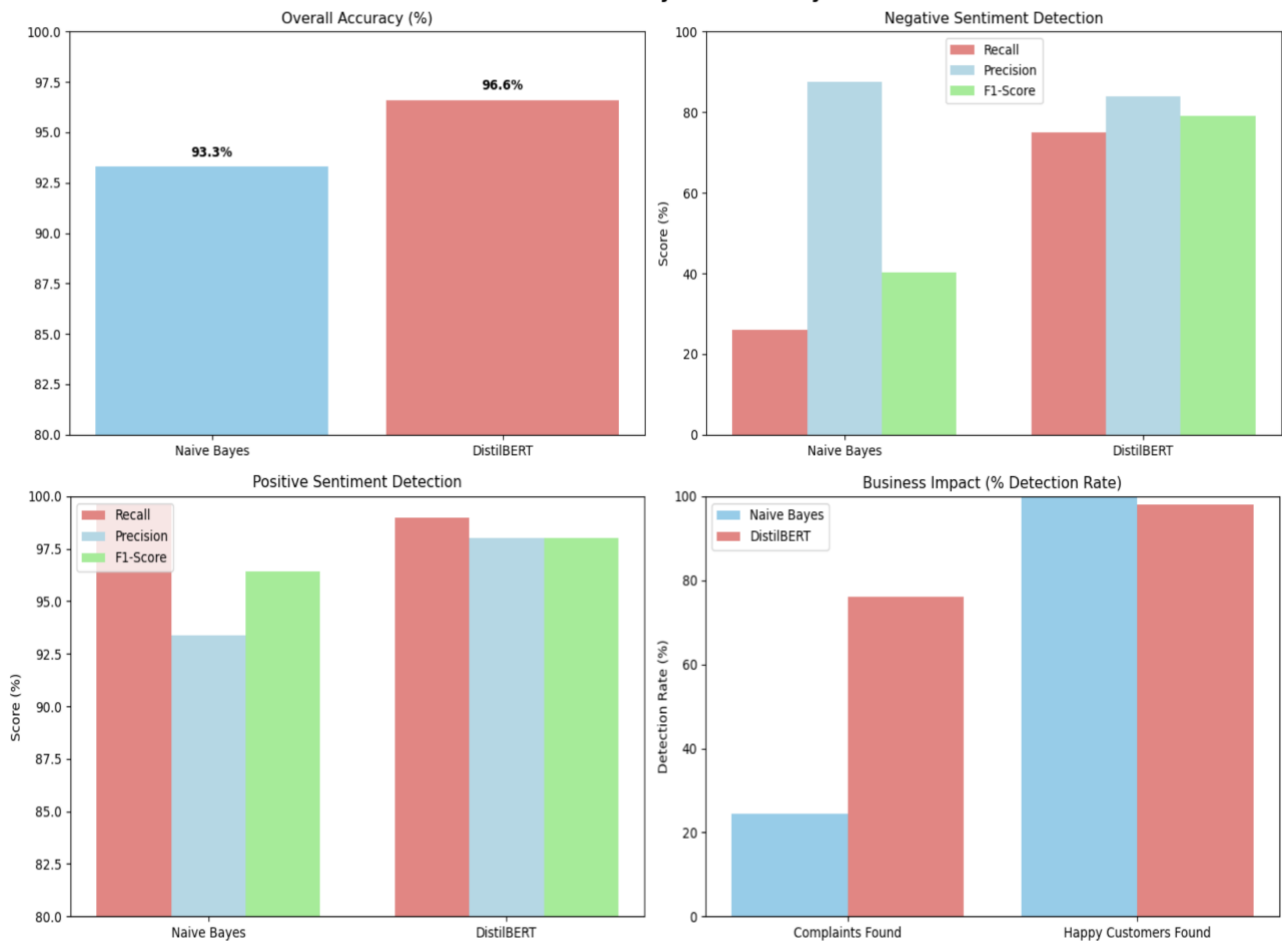
F1-Score: It did not do that badly with a rate of 79% overall.

Overall, BERT performed much better than the Naives Bayes model.

III. CONCLUSION

A. OVERALL PERFORMANCE COMPARISON

Universal Studios Sentiment Analysis: Naive Bayes vs DistilBERT



PERFORMANCE COMPARISON TABLE

Metric	Naive Bayes	DistilBERT	Improvement
Overall Accuracy	93.3	96.6	+3.2999999999999997

NEGATIVE REVIEWS METRICS

Metric	Naive Bayes	DistilBERT	Improvement
Negative Recall	26.1	75.0	+48.9
Negative Precision	87.6	84.0	-3.5999999999999994
Negative F1-Score	40.2	79.0	+38.8

POSITIVE REVIEWS METRICS

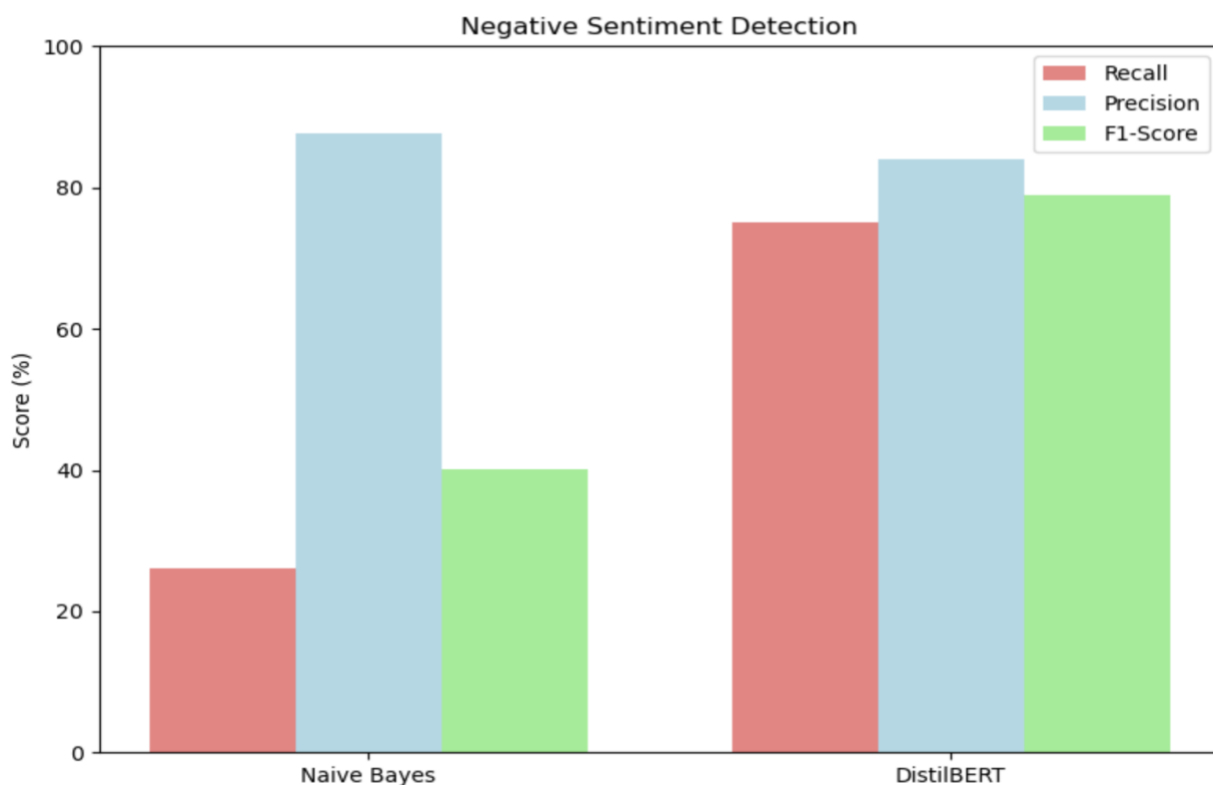
Metric	Naive Bayes	DistilBERT	Improvement
Positive Recall	99.6	99.0	-0.5999999999999994
Positive Precision	93.4	98.0	+4.5999999999999994
Positive F1-Score	96.4	98.0	+1.5999999999999994

BERT achieved an overall accuracy rate of 96.6% compared to Naive Bayes, which achieved 93.3%. However, this subtle difference does not show the big difference between the two models' capabilities at detecting negative reviews, even though they are both good at detecting positive ones. It is their approaches at handling minority class detection that show the disparity between the two models' performances.

B. CLASS-SPECIFIC PERFORMANCE ANALYSIS

NEGATIVE SENTIMENT DETECTION

Universal Studios Sentiment Analysis: Negative Sentiment Detection



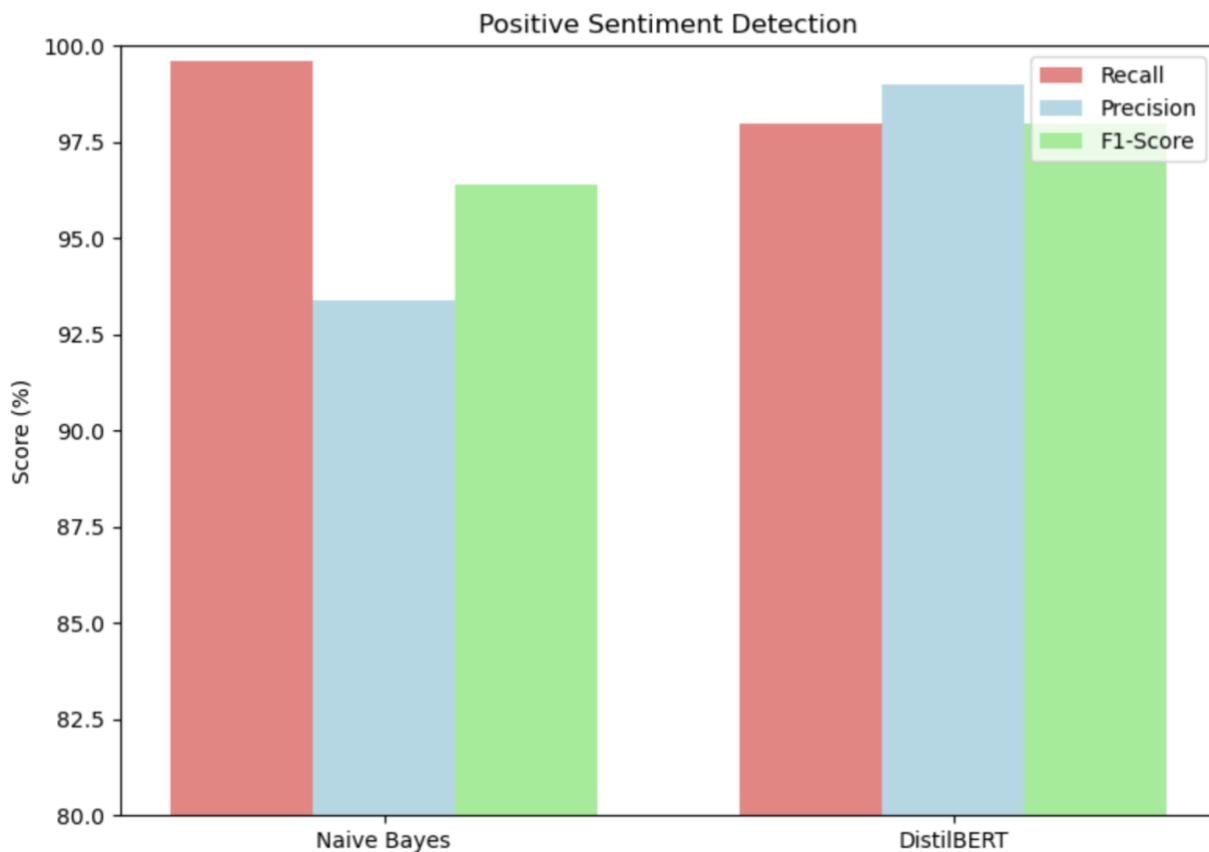
PERFORMANCE COMPARISON TABLE

Metric	Naive Bayes	DistilBERT	Improvement
Overall Accuracy	93.3	96.6	+3.299999999999997
Negative Recall	26.1	75.0	+48.9
Negative Precision	87.6	84.0	-3.5999999999999943
Negative F1-Score	40.2	79.0	+38.8

The BERT model significantly outperforms Naives Bayes, especially in the metric of negative sentiment recall, i.e., 75% vs. 26.1%. This shows that BERT can correctly identify 48.9% more customer complaints than Naives Bayes. However, the only consolation for Naives Bayes is that it has a slightly higher negative precision score of 87.6% to BERT's 84%, indicating few false alarms when flagging a negative review.

POSITIVE SENTIMENT DETECTION

Universal Studios Sentiment Analysis: Positive Sentiment Detection



POSITIVE REVIEWS METRICS

Metric	Naive Bayes	DistilBERT	Improvement
Positive Recall	99.6	98.0	-1.5999999999999943
Positive Precision	93.4	99.0	+5.5999999999999994
Positive F1-Score	96.4	98.0	+1.5999999999999943

Both models performed exceptionally well at detecting positive reviews, with the Naives Bayes achieving a marginally better recall score of 99.6% to BERT's 98%, while BERT showed better precision at 99% to Naives Bayes' 93.4%. The F1-scores show that BERT's performance is more balanced than Naive Bayes' overall.

SAMPLES OF INCORRECTLY LABELLED SENTIMENTS BY NAIVES BAYES

PERFORMANCE BREAKDOWN:

```
=====
Both Correct   :    6294 (91.9%)
BERT Better    :     327 (4.8%)
Both Wrong     :     134 (2.0%)
NB Better      :      97 (1.4%)
=====
```

CRITICAL FAILURES: Negative reviews Naive Bayes missed but BERT caught

BERT BETTER EXAMPLES:

Example 1:

TRUE SENTIMENT: NEGATIVE

Naive Bayes predicted: POSITIVE

BERT predicted: NEGATIVE

Review: Our first trip to HHN was last year of 2013. My husband and I were thrilled at the non stop scaring at every corner. We decided to come again this year but with our family.. and we were extremely disappointed. The lines were ridiculous for the houses and the scare zones were not scary. To come from the whole park flooded with zombies to this.. Was pretty upsetting. Also, it completely disgusts me ...

Example 2:

TRUE SENTIMENT: NEGATIVE

Naive Bayes predicted: POSITIVE

BERT predicted: NEGATIVE

Review: Very small park, 2 hour queues for rides and other guests don't know how to queue! Lots of pushing and shoving in queues and stalking in eateries for your seats, actually had a stranger join us at our booth waiting for us to vacate!...

Example 3:

TRUE SENTIMENT: NEGATIVE

Naive Bayes predicted: POSITIVE

BERT predicted: NEGATIVE

Review: Visited the first time in Japan, small theme park, not much activity, all roller coaster ride, went to Harry Potter land and spider s Man ride, Terminator show was closed early, shows were only in Japanese language, all restaurants were expensive, went to the shops, most of the stuff were not interested and so expensive, found one t-shirt but no extra large, size only small to Large. Overall, very ...

Example 4:

TRUE SENTIMENT: NEGATIVE

Naive Bayes predicted: POSITIVE

BERT predicted: NEGATIVE

Review: Very disappointing. I love theme parks but this just isn't in the same class as those in the UK (Alton Towers) or US (Six Flags or the original Universal Studios in Florida). For starters it's a very small park. Very few major rides and they are all toned down. The biggest roller coaster here, Battlestar Gallactica has been under maintenance for several months and will be so for many more months. ...

Example 5:

TRUE SENTIMENT: NEGATIVE

Naive Bayes predicted: POSITIVE

BERT predicted: NEGATIVE

Review: Recently went Disney in Hong Kong and loved it so thought I'd give this a try. What a mistake that was! Staff members just not interested and just counting down the hours till they can go home. No atmosphere, no fun vibe. Food was hugely overpriced and not nice. I know what prices to expect, but Disney was a better experience all round. Extra charges everywhere, gone are the days where you can p...

C. STATISTICAL VS. EMBEDDING-BASED MODEL

Naives Bayes, representing the statistical approach, excelled over BERT's embedding-based approach in its computational efficiency, interpretability, and especially its training speed; i.e., Naives Bayes took less than a second, whereas BERT took a whole 39.9 minutes to finish its training process, not to mention the computational memory it took from the hardware. Also, its feature-based approach (i.e., tokens from the text) provides transparent decision-making through calculating the likelihood of certain words appearing in positive or negative contexts. Its performance at detecting positive sentiments is commendable, along with minimal false negative flags for complaints.

BERT, on the other hand, has superior contextual understanding (i.e., no need to remove punctuation or stopwords) that allows it to handle complex sentiment expressions and nuanced language patterns. Its transformer design allows it to understand the meaning and context of words across an entire review, not just how often words appear. This helps it detect negative sentiments much more accurately than older methods that rely only on word frequency.

However, in the context of real-world scenarios and choice of model, it depends on what the critical needs of the company are.

For **Universal Studios**, if the primary concern is **operational efficiency and real-time guest feedback monitoring during peak seasons**, Naive Bayes offers immediate processing capabilities and transparent decision-making that operations teams can quickly understand and act upon. **This approach suits resource-constrained environments or when rapid deployment across multiple park locations is essential.**

Conversely, if Universal Studios prioritizes **comprehensive customer experience management and can tolerate longer processing times**, DistilBERT's superior negative sentiment detection becomes invaluable, potentially preventing customers from getting upset by identifying and addressing complaints that would otherwise go unnoticed.

The choice ultimately depends on whether the organization values speed and simplicity over accuracy and depth, the computational infrastructure available, and the business cost of missing negative feedback versus the overhead of implementing more complex systems.

IV. SUMMARY & REFLECTION

A. LEARNING EXPERIENCE

This study has shown me the fundamental differences between statistical and embedding-based approaches to sentiment classification. The most significant learning was discovering how class imbalance dramatically impacts model performance differently: while Naive Bayes achieved impressive overall accuracy (93.3%), it failed at efficiently detecting negative sentiment (26.1% recall), whereas DistilBERT's contextual understanding enabled substantially better minority class detection (75% recall). This highlighted the critical importance of examining class-specific metrics rather than relying solely on aggregate accuracy measures in business applications.

B. MODEL PRACTICALITY

Naive Bayes demonstrates exceptional practicality for resource-constrained environments, providing training times measured in seconds, transparent decision-making through interpretable probability calculations, and minimal computational overhead suitable for real-time applications. DistilBERT, while requiring 39.9 minutes for training and significantly more computational resources, provides superior performance for complex sentiment analysis tasks requiring nuanced understanding of context, sarcasm, and mixed sentiment expressions.

C. APPLICATION & TRANSFERABILITY

The Universal Studios case study demonstrates clear transferability to numerous domains requiring sentiment analysis. On top of the tourism and entertainment industry, several other domains could also adapt the findings for their use: i.e., e-commerce platforms could leverage these findings for product review analysis, healthcare organizations for patient feedback assessment, and financial institutions for customer satisfaction monitoring. The methodology scales effectively to any domain with significant class imbalance challenges, particularly where missing negative feedback carries high business costs.

D. CONTRIBUTION & FUTURE RESEARCH

This analysis offers practical guidance for selecting sentiment analysis models, particularly for applications in the tourism and entertainment industries, as outlined in the introduction. It highlights the trade-offs between computational efficiency and detection accuracy, showing that while advanced models require more resources, their improved ability to detect customer complaints can have a significant impact on business outcomes—making the additional computational cost worthwhile. Future research could explore combining both models through ensemble methods to get the best of both worlds. It would also be useful to look into active learning techniques to better handle class imbalances in the data. Lastly, another direction is developing hybrid models that keep BERT's strong understanding of context but run more efficiently, like Naive Bayes.

REFERENCES

Bai, S., He, H., Han, C., Yang, M., Bi, X., & Fan, W. (2023). What makes a theme park experience less enjoyable? Evidence from online customer reviews of Disneyland China. *Frontiers in Psychology*, 14, Article 1120483. <https://doi.org/10.3389/fpsyg.2023.1120483>

Nurdialit, D. G. (2021). Reviews of Universal Studios [Data set]. Kaggle. <https://doi.org/10.34740/kaggle/dsv/2304311>

Gallagher, C., Furey, E., & Curran, K. (2019). The application of sentiment analysis and text analytics to customer experience reviews to understand what customers are really saying. *International Journal of Data Warehousing and Mining*, 15(4), 21–47. <https://doi.org/10.4018/IJDWM.2019100102>

Niu, Y., Park, H., & Kirilenko, A. (2019). Theme park visitor experience and satisfaction: A case of TripAdvisor reviews of three theme parks in Orlando. *Travel and Tourism Research Association: Advancing Tourism Research Globally*, 2019(11). https://scholarworks.umass.edu/ttra/2019/research_papers/11

CODE

DATA PREPROCESSING – DATA EXPLORATION

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv('universal_studio_branches.csv')

# Data exploration
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nFirst few rows:")
df.head()
```

CHECK FOR MISSING VALUES AND RATING DISTRIBUTION

```
# Check for missing values
print("\nMissing values:")
print(df.isnull().sum())

# Look at rating distribution
print("\nRating distribution:")
print(df['rating'].value_counts().sort_index())
```

BRANCH DISTRIBUTION

```
# Branch distribution
print("Branch distribution:")
print(df['branch'].value_counts())
```

REVIEW TEXT LENGTH

```
# Review text lengths
df['text_length'] = df['review_text'].str.len()
print("\nReview text length statistics:")
print(df['text_length'].describe())

# Sample review texts by rating
print("\nSample reviews by rating:")
for rating in [1, 3, 5]:
    print(f"\n--- Rating {rating} example ---")
    sample = df[df['rating'] == rating]['review_text'].iloc[0]
    print(sample[:200] + "..." if len(sample) > 200 else sample)
```

CREATE POSITIVE/NEGATIVE LABELS

```
# 1-2 as negative, 4-5 as positive, and skip 3 for now
df['sentiment'] = df['rating'].map({
    1.0: 'negative',
    2.0: 'negative',
    4.0: 'positive',
    5.0: 'positive'
})

# Remove neutral reviews (rating 3) for clearer sentiment classification
df_sentiment = df[df['rating'] != 3.0].copy()

print(f"After removing neutral reviews:")
print(f"Original dataset: {len(df)} reviews")
print(f"Sentiment dataset: {len(df_sentiment)} reviews")
print("\nSentiment distribution:")
print(df_sentiment['sentiment'].value_counts())
```

BASIC TEXT CLEANING

```
import re
import string

# Basic text cleaning function
def clean_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove special characters and digits
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    # Remove extra whitespace
    text = ' '.join(text.split())
    return text

# Apply cleaning to a sample
print("Before cleaning:")
```

```

print(df_sentiment['review_text'].iloc[0][:200])

# Clean the text
df_sentiment['clean_text'] = df_sentiment['review_text'].apply(clean_text)

print("\nAfter cleaning:")
print(df_sentiment['clean_text'].iloc[0][:200])

```

PREPROCESSING FOR NAIVES BAYES

REMOVAL OF STOPWORDS

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

# Preprocessing for Naive Bayes
import nltk
# nltk.download('stopwords')
from nltk.corpus import stopwords

# English stopwords
stop_words = set(stopwords.words('english'))

# Naive Bayes preprocessing function
def preprocess_for_nb(text):
    # Remove stopwords
    words = text.split()
    words = [word for word in words if word not in stop_words and len(word) >
2]
    return ' '.join(words)

# Apply NB preprocessing
df_sentiment['nb_text'] = df_sentiment['clean_text'].apply(preprocess_for_nb)

print("Naive Bayes preprocessing example:")
print("Original:", df_sentiment['clean_text'].iloc[0][:150])
print("NB version:", df_sentiment['nb_text'].iloc[0][:150])

```

TRAIN/VALIDATION/TEST SPLITS

```

# Train/Validation/Test splits (70/15/15)
X = df_sentiment['review_text']
y = df_sentiment['sentiment']

# First split: 70% train, 30% temp
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42, # for reproducibility
    stratify=y       # keeps same positive/negative ratio
)

# Second split: 30% temp → 15% validation, 15% test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.5, # 50% of 30% = 15% each
    random_state=42,
    stratify=y_temp
)

```

```

print("Train/Validation/Test Split Results:")
print(f"Training samples: {len(X_train)} ({len(X_train)/len(X)*100:.1f}%)")
print(f"Validation samples: {len(X_val)} ({len(X_val)/len(X)*100:.1f}%)")
print(f"Testing samples: {len(X_test)} ({len(X_test)/len(X)*100:.1f}%)")

print(f"\nTraining sentiment distribution:")
train_counts = y_train.value_counts()
print(train_counts)
for sentiment in ['positive', 'negative']:
    count = train_counts[sentiment]
    percentage = (count / len(y_train)) * 100
    print(f"{sentiment}: {count} ({percentage:.1f}%)")

print(f"\nValidation sentiment distribution:")
val_counts = y_val.value_counts()
print(val_counts)
for sentiment in ['positive', 'negative']:
    count = val_counts[sentiment]
    percentage = (count / len(y_val)) * 100
    print(f"{sentiment}: {count} ({percentage:.1f}%)")

print(f"\nTesting sentiment distribution:")
test_counts = y_test.value_counts()
print(test_counts)
for sentiment in ['positive', 'negative']:
    count = test_counts[sentiment]
    percentage = (count / len(y_test)) * 100
    print(f"{sentiment}: {count} ({percentage:.1f}%)")

```

CREATE TF-IDF VECTORS

```

# Get preprocessed text for train/validation/test sets
X_train_nb = df_sentiment.loc[X_train.index, 'nb_text']
X_val_nb = df_sentiment.loc[X_val.index, 'nb_text']
X_test_nb = df_sentiment.loc[X_test.index, 'nb_text']

# Create TF-IDF vectorizer
tfidf = TfidfVectorizer(
    max_features=5000,      # limit vocabulary to top 5000 words
    min_df=2,              # word must appear in at least 2 documents
    max_df=0.8             # ignore words in more than 80% of documents
)

# Fit on training data and transform all three sets
X_train_tfidf = tfidf.fit_transform(X_train_nb)
X_val_tfidf = tfidf.transform(X_val_nb)
X_test_tfidf = tfidf.transform(X_test_nb)

print("TF-IDF Vectorization Results:")
print(f"Vocabulary size: {len(tfidf.vocabulary_)}")
print(f"Training matrix shape: {X_train_tfidf.shape}")
print(f"Validation matrix shape: {X_val_tfidf.shape}")
print(f"Testing matrix shape: {X_test_tfidf.shape}")

# Example features
feature_names = tfidf.get_feature_names_out()
print(f"\nSample vocabulary words: {feature_names[:20].tolist()}")

```

IMPLEMENT NAIVES BAYES TO GET BASELINE

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report

# Create and train Naive Bayes model
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)

# Make predictions
y_pred_nb = nb_model.predict(X_test_tfidf)

# Calculate metrics
nb_accuracy = accuracy_score(y_test, y_pred_nb)
nb_precision = precision_score(y_test, y_pred_nb, pos_label='positive')
nb_recall = recall_score(y_test, y_pred_nb, pos_label='positive')
nb_f1 = f1_score(y_test, y_pred_nb, pos_label='positive')

print("Naive Bayes Results:")
print(f"Accuracy: {nb_accuracy:.3f}")
print(f"Precision: {nb_precision:.3f}")
print(f"Recall: {nb_recall:.3f}")
print(f"F1-Score: {nb_f1:.3f}")
```

NEGATIVE CLASS PERFORMANCE

```
# Negative class performance
from sklearn.metrics import precision_score, recall_score, f1_score

nb_precision_neg = precision_score(y_test, y_pred_nb, pos_label='negative')
nb_recall_neg = recall_score(y_test, y_pred_nb, pos_label='negative')
nb_f1_neg = f1_score(y_test, y_pred_nb, pos_label='negative')

print(f"\nNAIVE BAYES - NEGATIVE CLASS PERFORMANCE:")
print("="*50)
print(f"Precision (negative): {nb_precision_neg:.3f}")
print(f"Recall (negative): {nb_recall_neg:.3f}")
print(f"F1-Score (negative): {nb_f1_neg:.3f}")

# Business interpretation
total_negative_reviews = (y_test == 'negative').sum()
detected_negative_reviews = int(nb_recall_neg * total_negative_reviews)
missed_negative_reviews = total_negative_reviews - detected_negative_reviews

print(f"\nBUSINESS IMPACT:")
print(f"Total negative reviews: {total_negative_reviews}")
print(f"Detected by Naive Bayes: {detected_negative_reviews}")
print(f"Missed complaints: {missed_negative_reviews}")
print(f"Miss rate: {(1 - nb_recall_neg)*100:.1f}%")
```

IMPLEMENT NAIVES BAYES ON VALIDATION AND TEST SETS

```
# Test Naive Bayes on Validation Set
print("NAIVE BAYES - VALIDATION SET EVALUATION")
print("="*50)

# Get predictions on validation set
y_pred_nb_val = nb_model.predict(X_val_tfidf)

# Calculate validation metrics
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

nb_val_accuracy = accuracy_score(y_val, y_pred_nb_val)
print(f"Validation Accuracy: {nb_val_accuracy:.3f}")
print("\nValidation Classification Report:")
print(classification_report(y_val, y_pred_nb_val, target_names=['Negative',
'Positive']))

# Test Naive Bayes on Test Set
print("\nNAIVE BAYES - TEST SET EVALUATION")
print("="*50)

# Get predictions on test set
y_pred_nb_test = nb_model.predict(X_test_tfidf)

# Calculate test metrics
nb_test_accuracy = accuracy_score(y_test, y_pred_nb_test)
print(f"Test Accuracy: {nb_test_accuracy:.3f}")
print("\nTest Classification Report:")
print(classification_report(y_test, y_pred_nb_test, target_names=['Negative',
'Positive']))

# Compare Validation vs Test Performance
print("\nNAIVE BAYES - VALIDATION vs TEST COMPARISON")
print("="*50)
print(f"{'Metric':<20} {'Validation':<12} {'Test':<12} {'Difference':<12}")
print("-" * 56)
print(f"{'Accuracy':<20} {nb_val_accuracy:<12.3f} {nb_test_accuracy:<12.3f}
{abs(nb_val_accuracy - nb_test_accuracy):<12.3f}")

# Store results for later BERT comparison
nb_val_report = classification_report(y_val, y_pred_nb_val,
target_names=['Negative', 'Positive'], output_dict=True)
nb_test_report = classification_report(y_test, y_pred_nb_test,
target_names=['Negative', 'Positive'], output_dict=True)

print(f"\nValidation Negative Recall:
{nb_val_report['Negative']['recall']:.3f}")
print(f"Test Negative Recall: {nb_test_report['Negative']['recall']:.3f}")
```


SET UP BERT DATA SETS

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
Trainer, TrainingArguments
import torch
from torch.utils.data import Dataset
import numpy as np

print("BERT imports successful")

# Import Dataset from datasets library
from datasets import Dataset

# Disable widgets to prevent display errors
import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"
os.environ["DISABLE_TQDM"] = "true"

# Also disable datasets progress bars
import datasets
datasets.disable_progress_bar()

# Load BERT tokenizer
tokenizer = AutoTokenizer.from_pretrained('distilbert-base-uncased')

# Use basic clean text for all three sets
X_train_bert = df_sentiment.loc[X_train.index, 'clean_text'].tolist()
X_val_bert = df_sentiment.loc[X_val.index, 'clean_text'].tolist()
X_test_bert = df_sentiment.loc[X_test.index, 'clean_text'].tolist()

# Convert labels to numbers for BERT
label_map = {'negative': 0, 'positive': 1}
y_train_bert = [label_map[label] for label in y_train]
y_val_bert = [label_map[label] for label in y_val]
y_test_bert = [label_map[label] for label in y_test]

print("BERT Data Preparation:")
print(f"Training samples: {len(X_train_bert)}")
print(f"Validation samples: {len(X_val_bert)}")
print(f"Testing samples: {len(X_test_bert)}")
print(f"Sample training text: {X_train_bert[0][:100]}...")
print(f"Sample training label: {y_train_bert[0]} (0=negative, 1=positive)")

# Create datasets for BERT using HuggingFace Dataset
train_dataset = Dataset.from_dict({
    'text': X_train_bert,
    'labels': y_train_bert
})

val_dataset = Dataset.from_dict({
    'text': X_val_bert,
    'labels': y_val_bert
})

test_dataset = Dataset.from_dict({
    'text': X_test_bert,
    'labels': y_test_bert
})
```

```

print(f"\nBERT Datasets Created:")
print(f"Train dataset: {len(train_dataset)} samples")
print(f"Validation dataset: {len(val_dataset)} samples")
print(f"Test dataset: {len(test_dataset)} samples")

# Add tokenization to datasets
def tokenize_function(examples):
    return tokenizer(examples['text'], truncation=True, padding='max_length',
max_length=128)

train_dataset = train_dataset.map(tokenize_function, batched=True)
val_dataset = val_dataset.map(tokenize_function, batched=True)
test_dataset = test_dataset.map(tokenize_function, batched=True)

print("Datasets tokenized and ready for BERT evaluation")

```

SET UP BERT TRAINING ARUGMENTS AND START TRAINING

```

import warnings
warnings.filterwarnings('ignore')
import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"
os.environ["WANDB_DISABLED"] = "true"
os.environ["TRANSFORMERS_NO_ADVISORY_WARNINGS"] = "true"

# For the pin_memory warning
if torch.backends.mps.is_available():
    torch.mps.empty_cache()

import time

model_name = 'distilbert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=2)

# Training arguments
training_args = TrainingArguments(
    output_dir='./distilbert_results',
    num_train_epochs=2,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    warmup_steps=500,
    weight_decay=0.01,
    logging_steps=500,
    save_steps=1000,
    eval_steps=1000,
    disable_tqdm=True,
    report_to=[],
    logging_first_step=False,
)

# Create trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,

```

```

)

print("DistilBERT trainer setup complete")
print("Starting training...")
start_time = time.time()

# Start training
training_results = trainer.train()

end_time = time.time()
training_minutes = (end_time - start_time) / 60
print(f"DistilBERT training completed in {training_minutes:.1f} minutes!")

# Display training metrics
print("\nTRAINING METRICS:")
print("="*40)
print(f"Final training loss: {training_results.training_loss:.4f}")
print(f"Training steps completed: {training_results.global_step}")
print(f"Training samples per second: {training_results.metrics['train_samples_per_second']:.2f}")
print(f"Training steps per second: {training_results.metrics['train_steps_per_second']:.3f}")

```

EVALUATE BERT MODEL ON VALIDATION AND TEST SETS

```

# Evaluate BERT model on Validation and Test sets
print("EVALUATING DISTILBERT ON VALIDATION AND TEST SETS")
print("="*60)

# =====
# VALIDATION SET EVALUATION
# =====
print("\nVALIDATION SET EVALUATION:")
print("-" * 40)

# Get predictions on validation set
predictions_val = trainer.predict(val_dataset)
y_pred_bert_val = predictions_val.predictions.argmax(-1)

# Convert pandas series string labels to integers for comparison
y_val_numeric = y_val.map({'negative': 0, 'positive': 1})

print("Validation label conversion check:")
print("Original y_val sample:", y_val.head())
print("Converted y_val_numeric sample:", y_val_numeric.head())
print("BERT validation predictions sample:", y_pred_bert_val[:5])

# Calculate validation metrics
bert_val_accuracy = accuracy_score(y_val_numeric, y_pred_bert_val)
bert_val_report = classification_report(y_val_numeric, y_pred_bert_val,
target_names=['Negative', 'Positive'], output_dict=True)

print(f"\nDistilBERT Validation Accuracy: {bert_val_accuracy:.3f}")
print("\nValidation Classification Report:")
print(classification_report(y_val_numeric, y_pred_bert_val,
target_names=['Negative', 'Positive']))

# =====
# TEST SET EVALUATION

```

```

# =====
print("\nTEST SET EVALUATION:")
print("-" * 40)

# Get predictions on test set
predictions_test = trainer.predict(test_dataset)
y_pred_bert_test = predictions_test.predictions.argmax(-1)

# Convert pandas series string labels to integers for comparison
y_test_numeric = y_test.map({'negative': 0, 'positive': 1})

print("Test label conversion check:")
print("Original y_test sample:", y_test.head())
print("Converted y_test_numeric sample:", y_test_numeric.head())
print("BERT test predictions sample:", y_pred_bert_test[:5])

# Calculate test metrics
bert_test_accuracy = accuracy_score(y_test_numeric, y_pred_bert_test)
bert_test_report = classification_report(y_test_numeric, y_pred_bert_test,
target_names=['Negative', 'Positive'], output_dict=True)

print(f"\nDistilBERT Test Accuracy: {bert_test_accuracy:.3f}")
print("\nTest Classification Report:")
print(classification_report(y_test_numeric, y_pred_bert_test,
target_names=['Negative', 'Positive']))

# =====
# VALIDATION vs TEST COMPARISON
# =====
print("\nDISTILBERT - VALIDATION vs TEST COMPARISON")
print("="*60)
print(f"{'Metric':<25} {'Validation':<12} {'Test':<12} {'Difference':<12}")
print("-" * 61)
print(f"{'Accuracy':<25} {bert_val_accuracy:<12.3f} {bert_test_accuracy:<12.3f} {abs(bert_val_accuracy - bert_test_accuracy):<12.3f}")
print(f"{'Negative Recall':<25} {bert_val_report['Negative']['recall']:<12.3f} {bert_test_report['Negative']['recall']:<12.3f} {abs(bert_val_report['Negative']['recall'] - bert_test_report['Negative']['recall']):<12.3f}")
print(f"{'Positive Recall':<25} {bert_val_report['Positive']['recall']:<12.3f} {bert_test_report['Positive']['recall']:<12.3f} {abs(bert_val_report['Positive']['recall'] - bert_test_report['Positive']['recall']):<12.3f}")

```

OVERALL PERFORMANCE COMPARISON CHARTS

```

import matplotlib.pyplot as plt
import numpy as np

# Performance data
models = ['Naive Bayes', 'DistilBERT']
accuracy = [93.3, 96.6]
negative_recall = [26.1, 75.0]
negative_precision = [87.6, 84.0]
negative_f1 = [40.2, 79.0]
positive_recall = [99.6, 99.0]

```

```

positive_precision = [93.4, 98.0]
positive_f1 = [96.4, 98.0]

# Create comparison charts
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Universal Studios Sentiment Analysis: Naive Bayes vs
DistilBERT', fontsize=16, fontweight='bold')

# Overall Accuracy
axes[0,0].bar(models, accuracy, color=['skyblue', 'lightcoral'])
axes[0,0].set_title('Overall Accuracy (%)')
axes[0,0].set_ylim(80, 100)
for i, v in enumerate(accuracy):
    axes[0,0].text(i, v + 0.5, f'{v}%', ha='center', fontweight='bold')

# Negative Class Performance
x = np.arange(len(models))
width = 0.25
axes[0,1].bar(x - width, negative_recall, width, label='Recall',
color='lightcoral')
axes[0,1].bar(x, negative_precision, width, label='Precision',
color='lightblue')
axes[0,1].bar(x + width, negative_f1, width, label='F1-Score',
color='lightgreen')
axes[0,1].set_title('Negative Sentiment Detection')
axes[0,1].set_ylabel('Score (%)')
axes[0,1].set_xticks(x)
axes[0,1].set_xticklabels(models)
axes[0,1].legend()
axes[0,1].set_ylim(0, 100)

# Positive Class Performance
axes[1,0].bar(x - width, positive_recall, width, label='Recall',
color='lightcoral')
axes[1,0].bar(x, positive_precision, width, label='Precision',
color='lightblue')
axes[1,0].bar(x + width, positive_f1, width, label='F1-Score',
color='lightgreen')
axes[1,0].set_title('Positive Sentiment Detection')
axes[1,0].set_ylabel('Score (%)')
axes[1,0].set_xticks(x)
axes[1,0].set_xticklabels(models)
axes[1,0].legend()
axes[1,0].set_ylim(80, 100)

# Key Business Metrics
business_metrics = ['Complaints Found', 'Happy Customers Found']
nb_business = [24.5, 99.6] # negative recall, positive recall
bert_business = [76.0, 98.0]
x_business = np.arange(len(business_metrics))
axes[1,1].bar(x_business - 0.2, nb_business, 0.4, label='Naive Bayes',
color='skyblue')
axes[1,1].bar(x_business + 0.2, bert_business, 0.4, label='DistilBERT',
color='lightcoral')
axes[1,1].set_title('Business Impact (% Detection Rate)')
axes[1,1].set_ylabel('Detection Rate (%)')
axes[1,1].set_xticks(x_business)
axes[1,1].set_xticklabels(business_metrics)
axes[1,1].legend()
axes[1,1].set_ylim(0, 100)

```

```

plt.tight_layout()
plt.show()

# Summary tables
print("PERFORMANCE COMPARISON TABLE")
print("="*60)
print(f"{'Metric':<25} {'Naive Bayes':<15} {'DistilBERT':<15} {'Improvement':<15}")
print("="*60)
print(f"{'Overall Accuracy':<25} {accuracy[0]:<15} {accuracy[1]:<15} {'+' + str(accuracy[1] - accuracy[0]):<15}")
print("="*60)

print("\nNEGATIVE REVIEWS METRICS")
print("="*60)
print(f"{'Metric':<25} {'Naive Bayes':<15} {'DistilBERT':<15} {'Improvement':<15}")
print("="*60)
print(f"{'Negative Recall':<25} {negative_recall[0]:<15} {negative_recall[1]:<15} {'+' + str(negative_recall[1] - negative_recall[0]):<15}")
print(f"{'Negative Precision':<25} {negative_precision[0]:<15} {negative_precision[1]:<15} {'+' + str(negative_precision[1] - negative_precision[0]):<15}")
print(f"{'Negative F1-Score':<25} {negative_f1[0]:<15} {negative_f1[1]:<15} {'+' + str(negative_f1[1] - negative_f1[0]):<15}")
print("="*60)

print("\nPOSITIVE REVIEWS METRICS")
print("="*60)
print(f"{'Metric':<25} {'Naive Bayes':<15} {'DistilBERT':<15} {'Improvement':<15}")
print("="*60)
print(f"{'Positive Recall':<25} {positive_recall[0]:<15} {positive_recall[1]:<15} {'+' + str(positive_recall[1] - positive_recall[0]):<15}")
print(f"{'Positive Precision':<25} {positive_precision[0]:<15} {positive_precision[1]:<15} {'+' + str(positive_precision[1] - positive_precision[0]):<15}")
print(f"{'Positive F1-Score':<25} {positive_f1[0]:<15} {positive_f1[1]:<15} {'+' + str(positive_f1[1] - positive_f1[0]):<15}")
print("="*60)

```

CLASS-SPECIFIC PERFORMANCE ANALYSIS – NEGATIVE SENTIMENT DETECTION

```

import matplotlib.pyplot as plt
import numpy as np

# Performance data
models = ['Naive Bayes', 'DistilBERT']
negative_recall = [26.1, 75.0]
negative_precision = [87.6, 84.0]
negative_f1 = [40.2, 79.0]

# Create a single chart for negative sentiment detection
fig, ax = plt.subplots(figsize=(8, 6))
fig.suptitle('Universal Studios Sentiment Analysis: Negative Sentiment Detection', fontsize=14, fontweight='bold')

```

```

# Bar positions
x = np.arange(len(models))
width = 0.25

# Plot bars
ax.bar(x - width, negative_recall, width, label='Recall', color='lightcoral')
ax.bar(x, negative_precision, width, label='Precision', color='lightblue')
ax.bar(x + width, negative_f1, width, label='F1-Score', color='lightgreen')

# Styling
ax.set_title('Negative Sentiment Detection')
ax.set_ylabel('Score (%)')
ax.set_xticks(x)
ax.set_xticklabels(models)
ax.set_ylim(0, 100)
ax.legend()

plt.tight_layout()
plt.show()

# Summary table
print("PERFORMANCE COMPARISON TABLE")
print("="*60)
print(f"{'Metric':<25} {'Naive Bayes':<15} {'DistilBERT':<15} {'Improvement':<15}")
print("="*60)
print(f"{'Overall Accuracy':<25} {accuracy[0]:<15} {accuracy[1]:<15} {'+' + str(accuracy[1] - accuracy[0]):<15}")
print(f"{'Negative Recall':<25} {negative_recall[0]:<15} {negative_recall[1]:<15} {'+' + str(negative_recall[1] - negative_recall[0]):<15}")
print(f"{'Negative Precision':<25} {negative_precision[0]:<15} {negative_precision[1]:<15} {'+' + str(negative_precision[1] - negative_precision[0]):<15}")
print(f"{'Negative F1-Score':<25} {negative_f1[0]:<15} {negative_f1[1]:<15} {'+' + str(negative_f1[1] - negative_f1[0]):<15}")
print("="*60)

```

CLASS-SPECIFIC PERFORMANCE ANALYSIS – POSITIVE SENTIMENT DETECTION

```

import matplotlib.pyplot as plt
import numpy as np

# Performance data
models = ['Naive Bayes', 'DistilBERT']
positive_recall = [99.6, 98.0]
positive_precision = [93.4, 99.0]
positive_f1 = [96.4, 98.0]

# Create a single chart for positive sentiment detection
fig, ax = plt.subplots(figsize=(8, 6))
fig.suptitle('Universal Studios Sentiment Analysis: Positive Sentiment Detection', fontsize=14, fontweight='bold')

# Bar positions
x = np.arange(len(models))
width = 0.25

# Plot bars
ax.bar(x - width, positive_recall, width, label='Recall', color='lightcoral')

```

```

ax.bar(x, positive_precision, width, label='Precision', color='lightblue')
ax.bar(x + width, positive_f1, width, label='F1-Score', color='lightgreen')

# Styling
ax.set_title('Positive Sentiment Detection')
ax.set_ylabel('Score (%)')
ax.set_xticks(x)
ax.set_xticklabels(models)
ax.set_ylim(80, 100)
ax.legend()

plt.tight_layout()
plt.show()

print("\nPOSITIVE REVIEWS METRICS")
print("="*60)
print(f"{'Metric':<25} {'Naive Bayes':<15} {'DistilBERT':<15} {'Improvement':<15}")
print("="*60)
print(f"{'Positive Recall':<25} {positive_recall[0]:<15} {positive_recall[1]:<15} {str(positive_recall[1] - positive_recall[0]):<15}")
print(f"{'Positive Precision':<25} {positive_precision[0]:<15} {positive_precision[1]:<15} {'+' + str(positive_precision[1] - positive_precision[0]):<15}")
print(f"{'Positive F1-Score':<25} {positive_f1[0]:<15} {positive_f1[1]:<15} {'+' + str(positive_f1[1] - positive_f1[0]):<15}")
print("="*60)

```

SAMPLES OF INCORRECTLY LABELLED NEGATIVE REVIEWS

```

# Get Naive Bayes predictions on the same test set
y_pred_nb = nb_model.predict(X_test_tfidf)

# Convert Naive Bayes string predictions to integers to match BERT format
y_pred_nb_numeric = pd.Series(y_pred_nb).map({'negative': 0, 'positive': 1}).values

print("PREDICTION FORMATS:")
print(f"Naive Bayes original: {y_pred_nb[:5]}")
print(f"Naive Bayes converted: {y_pred_nb_numeric[:5]}")
print(f"BERT predictions: {y_pred_bert[:5]}")
print(f"True labels: {y_test_numeric.values[:5]}")

# Convert to numpy arrays to avoid index issues
y_true_array = y_test_numeric.values
y_nb_array = y_pred_nb_numeric
y_bert_array = y_pred_bert

# Create comparison dataframe with consistent numpy arrays
comparison_df = pd.DataFrame({
    'review_text': X_test.values if hasattr(X_test, 'values') else X_test,
    'true_label': y_true_array,
    'nb_prediction': y_nb_array,
    'bert_prediction': y_bert_array,
    'nb_correct': y_nb_array == y_true_array,
    'bert_correct': y_bert_array == y_true_array
})

# Add prediction categories

```



```

comparison_df['category'] = 'Both Correct'
comparison_df.loc[(comparison_df['nb_correct'] == False) &
(comparison_df['bert_correct'] == True), 'category'] = 'BERT Better'
comparison_df.loc[(comparison_df['nb_correct'] == True) &
(comparison_df['bert_correct'] == False), 'category'] = 'NB Better'
comparison_df.loc[(comparison_df['nb_correct'] == False) &
(comparison_df['bert_correct'] == False), 'category'] = 'Both Wrong'

print("\n PERFORMANCE BREAKDOWN:")
print("="*50)
category_counts = comparison_df['category'].value_counts()
for category, count in category_counts.items():
    percentage = (count / len(comparison_df)) * 100
    print(f"{category:<15}: {count:>6} ({percentage:.1f}%)")
print("="*50)

# Function to show review examples
def show_examples(category, num_examples=3, focus_class=None):
    if focus_class is not None:
        subset = comparison_df[(comparison_df['category'] == category) &
                                (comparison_df['true_label'] == focus_class)]
    else:
        subset = comparison_df[comparison_df['category'] == category]

    if len(subset) == 0:
        print(f"No examples found for {category}")
        return

    print(f"\n{category.upper()} EXAMPLES:")
    print("="*80)

    for i, (idx, row) in enumerate(subset.head(num_examples).iterrows()):
        label_map = {0: 'NEGATIVE', 1: 'POSITIVE'}
        true_sentiment = label_map[row['true_label']]
        nb_pred = label_map[row['nb_prediction']]
        bert_pred = label_map[row['bert_prediction']]

        print(f"\nExample {i+1}:")
        print(f"TRUE SENTIMENT: {true_sentiment}")
        print(f"Naive Bayes predicted: {nb_pred}")
        print(f"BERT predicted: {bert_pred}")
        print(f"Review: {row['review_text'][:400]}...")
        print("-" * 80)

# Show the most interesting cases
print("\nCRITICAL FAILURES: Negative reviews Naive Bayes missed but BERT
caught")
show_examples('BERT Better', num_examples=5, focus_class=0)

```

**** END ****